CO4 – Assignment 4

Complete solutions, C programs and sample outputs

Note: The answers follow the six tasks in the supplied assignment. Outputs shown are sample outputs for the stated inputs/compiler assumptions. Compile each complete C program separately.

Compilation: Use a C11-compatible compiler, for example: `gcc -std=c11 file.c -o program`.

TASK 1 – DATA DESIGN REVIEW

1.1 Diagnose the original design

1. Parallel arrays: A patient's name, age, phone, BP and date are stored at matching indexes. Inserting, deleting or sorting one array without moving every other array identically can mix one patient's details with another patient's record.

2. Phone field is too small: `char phone[100][10]` stores at most 9 characters plus the terminating '\0'. A 10-digit phone number needs 11 characters. The last digit may be lost or a string copy may write beyond the array.

3. BP stored in one float: Blood pressure has two separate values (systolic and diastolic), such as 120 and 80. A single float cannot represent and validate them independently; calculations and clinical comparisons become unreliable.

4. Date stored as text: An 8-character date has no separators and no numeric day/month/year fields. Invalid dates can be stored and chronological sorting/comparison is awkward.

5. No capacity check: When `count` reaches 100, writing to index 100 or beyond accesses memory outside the arrays, causing undefined behaviour and possible corruption.

6. Global state: Global arrays and count can be changed by any function. This makes errors harder to locate and functions harder to reuse or test.

7. Sorting difficulty: Sorting by age requires moving every parallel array element together. Missing one field during a swap breaks the association between patient data.

1.2 Repaired structure design and addPatient

```
#include <stdio.h>
#include <string.h>

#define MAX_PATIENTS 100

typedef struct { int day, month, year; } Date;
typedef struct { int systolic, diastolic; } BloodPressure;

typedef struct {
    char name[100];
    int age;
    char phone[11];       /* 10 digits + '\0' */
    BloodPressure bp;
    Date visitDate;
} Patient;

int addPatient(Patient patients[], int *count, const Patient *newPatient)
{
    if (*count >= MAX_PATIENTS) return 0;
    patients[*count] = *newPatient;
    ++(*count);
    return 1;
}

int main(void)
{
    Patient patients[MAX_PATIENTS];
    int count = 0;
    Patient p = {"Anu", 35, "9876543210", {120, 80}, {11, 11, 2026}};

    if (addPatient(patients, &count, &p))
        printf("Added: %s, age %d, BP %d/%d\n",
               patients[0].name, patients[0].age,
               patients[0].bp.systolic, patients[0].bp.diastolic);
    else
        printf("Patient register is full\n");
    return 0;
}
```

Sample output

```
Added: Anu, age 35, BP 120/80
```

1.3 Design defence (under 100 words)

An array of structures keeps every patient's fields together, so a record can be moved or sorted as one unit without accidentally separating related data. A function can receive one Patient pointer instead of many parallel-array parameters. Nested Date and BloodPressure structures make the meaning of each value clear and allow those types to be reused. New fields can be added to Patient in one place. The capacity check prevents writing beyond the register.

Alternative concept and complexity

Parallel arrays can be preferred when processing one field at a time, when a column-oriented layout improves cache use, or when only selected columns are needed. For a swap, an array-of-structures design moves one whole Patient record (roughly `sizeof(Patient)` bytes). Parallel arrays move each corresponding field separately; the total payload moved for one record swap is approximately the sum of the field sizes, though implementation and cache behaviour differ. A comparison sort is typically O(n log n); simple selection/bubble sorting is $O(n^2)$. Both layouts store O(n) data. Structures may contain padding for alignment; exact size is compiler-dependent and can be checked with `sizeof`. Parallel arrays may have less inter-field padding but still have alignment/padding within individual element types.

TASK 2 – FROM REQUIREMENT TO NESTED STRUCTURES

2.1 Entities and nesting

Parcel contains parcel ID, weight, booking date, sender, receiver, service type and charge. **Person/Address** data for sender and receiver contains name, city and PIN code. **Date** contains day, month and year. Address and Date are nested inside Parcel; sender and receiver each use the Address type.

2.2 Structure definitions

```
#include <stdio.h>
#include <string.h>

typedef struct { int day, month, year; } Date;
typedef struct {
    char name[40];
    char city[20];
    int pincode;
} PersonAddress;

typedef enum { NORMAL, EXPRESS } ServiceType;

typedef struct {
    int id;
    float weight;
    Date booked;
    PersonAddress sender;
    PersonAddress receiver;
    ServiceType service;
    float charge;
} Parcel;

Parcel bookings[100];
```

2.3 Operations – pseudocode

(a) Sort by weight: For i from 0 to n–2, set minIndex=i. Scan j from i+1 to n–1; if parcel[j].weight is less than parcel[minIndex].weight, update minIndex. Swap the complete Parcel records at i and minIndex. This selection sort takes $O(n^2)$ comparisons and O(1) extra storage.

(b) List by receiver city: For each parcel, compare receiver.city with the requested city (case-sensitive string comparison). If equal, print that parcel. This takes O(n) comparisons.

2.4 Access paths

(i) Array and dot operator: `list[2].to.pincode` for the assignment's supplied structure. In the expanded model above: `bookings[2].receiver.pincode`.

(ii) Pointer and arrow operator: `p->to.pincode` when p points to the assignment's parcel. In the expanded model: `p->receiver.pincode`.

Completed code (assignment's exact parcel model)

```
#include <stdio.h>

struct date { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel { int id; float weight; struct address to; struct date booked; };

void print_parcel(const struct parcel *p)
{
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight) min = j;
        if (min != i) {
            struct parcel temp = p[i];
            p[i] = p[min];
            p[min] = temp;
        }
    }
}

int main(void)
{
    struct parcel list[3] = {
```

```
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);
    return 0;
}
```

Expected output

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
```

Each swap uses a temporary whole structure and performs structure assignments. The nominal record size is `sizeof(struct parcel)` bytes; the three assignments in the swap may involve compiler-generated copying. The exact byte count is implementation-dependent.

Flat versus nested design

A flat version places city, PIN, day, month and year directly in struct parcel. Its access becomes `p->city` and `p->day`, rather than `p->to.city` and `p->booked.day`. Nested structures improve reuse and group related fields. Total size may be equal or differ because of padding; verify both with `sizeof` on the target compiler.

TASK 3 – UNIONS AND MEMORY LAYOUT

3.1 Structure versus tagged union

```
/* Stores all values simultaneously */
struct all_values {
    float celsius;
    int percent;
    char status[8];
};

/* Stores one active value, identified by type */
enum kind { TEMPERATURE, HUMIDITY, STATUS };
struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};
```

The tagged union is suitable because each packet carries exactly one value. The union shares storage among its members, reducing memory use. The tag records which member is currently valid.

3.2 Predicted size

On the typical 64-bit GCC/ABI assumed by the assignment, the union is 8 bytes because its largest member is `char status[8]`. The enum tag commonly occupies 4 bytes. The union needs alignment, and the structure size is rounded to a multiple of the strictest member alignment. Thus 4-byte tag + 8-byte union = 12 bytes on this target. The all-fields structure is commonly 16 bytes (4 + 4 + 8), subject to ABI rules. Always verify using sizeof; sizes are not guaranteed identical on every platform.

3.3 Completed program

```
#include <stdio.h>
enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};

void show(const struct reading *r)
{
    switch (r->type) {
    case TEMPERATURE:
        printf("Temp %.1f C\n", r->value.celsius);
        break;
    case HUMIDITY:
        printf("Humidity %d %%\n", r->value.percent);
        break;
    case STATUS:
        printf("Status %s\n", r->value.status);
        break;
    }
}

int main(void)
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    for (int i = 0; i < 3; i++) show(&log[i]);
    return 0;
}
```

Expected output on the assignment's typical 64-bit GCC system

```
sizeof union = 8, sizeof struct reading = 12
Temp 31.5 C
Humidity 68 %
Status OK
```

3.4 Misuse experiment

If the temperature member is written and the humidity member is read, the tag still says TEMPERATURE but the code reads a different union member. The resulting integer is not a meaningful humidity value; its interpretation is implementation-dependent and must not be used as valid sensor data. Always check the tag before reading the union member.

Alternative designs and complexity

Three separate arrays can store temperatures, humidity values and status strings efficiently when packet types are processed in batches. However, a packet index/type mapping is needed and combining the arrays into original arrival order is more complex. A tagged union gives O(1) access to a packet and uses one tag plus the largest union member, with possible padding. A malloc/void-pointer design can allocate only the needed payload but requires allocation metadata, casts, explicit type tracking and free calls; allocation/deallocation adds overhead and risks leaks. Reading a known packet is O(1) in all three designs.

TASK 4 – STRUCTURE FORENSICS

4.1 Defects

The function receives `struct account a` by value, so it changes only a local copy. The original array element remains unchanged. In main, p is a pointer, so `p.number` and `p.balance` are invalid member access; use `p->number` and `p->balance` (or `(*p).number`).

4.2 Address evidence

Print &a inside the by-value function and `&list[i]` in main. They will normally be different addresses because a is a separate parameter copy. Exact addresses vary on each run.

4.3 Corrected program

```
#include <stdio.h>

struct account { int number; float balance; };

void addInterest(struct account *a, float rate)
{
    printf("Address inside function: %p\n", (void *)a);
    a->balance += a->balance * rate / 100.0f;
}

int main(void)
{
    struct account list[3] = {
        {101, 5000}, {102, 12000}, {103, 800}
    };
    for (int i = 0; i < 3; i++) {
        printf("Address in main: %p\n", (void *)&list[i]);
        addInterest(&list[i], 4);
    }
    struct account *p = list;
    printf("%d %.2f\n", p->number, p->balance);
    for (int i = 0; i < 3; i++)
        printf("%d %.2f\n", list[i].number, list[i].balance);
    return 0;
}
```

Sample output (addresses will differ)

```
Address in main: 0x...
Address inside function: 0x...   (same address as corresponding list[i])
(repeated for three accounts)
101 5200.00
102 12480.00
103 832.00
```

4.4 Passing guideline

By value: Use when the function needs a small independent copy and must not modify the caller's object. **By pointer:** Use when the function must modify the original structure or avoid copying a large structure; check for NULL when appropriate. **By const pointer:** Use when the function only reads the structure and copying should be avoided; const prevents modification through that pointer.

Alternative: return the modified copy

```
struct account addInterestValue(struct account a, float rate)
{
    a.balance += a.balance * rate / 100.0f;
    return a;
}
/* caller: list[i] = addInterestValue(list[i], 4); */
```

Returning by value copies a structure into the parameter and returns the modified value (the compiler may optimize copies). It uses O(1) time for a fixed-size account and temporary stack/return storage, depending on the ABI. Pointer passing transfers only an address and updates the original directly. Both approaches use O(1) extra space per call; pointer passing is generally preferable when in-place modification is required.

TASK 5 – STORAGE CLASSES: THE SAME COUNTER FOUR WAYS

5.1 Implementations and expected tokens

```
#include <stdio.h>

/* (b) Static local: retains value between calls */
int next_token(void)
{
    static int token = 1000;
    return ++token;
}

/* (a) Automatic local: starts again on every call */
int auto_token(void)
{
    int token = 1000;
    return ++token;
}

/* (c) Global shared across source files:
   file1.c: int shared_token = 1000;
   file1.c or file2.c:
       extern int shared_token;
       int global_next_token(void) { return ++shared_token; }
*/

/* (d) Register loop variable */
void print_tokens(void)
{
    for (register int i = 1001; i <= 1003; ++i)
        printf("%d ", i);
    printf("\n");
}

int main(void)
{
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();
    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto : %d %d %d\n", a1, a2, a3);
    print_tokens();
    return 0;
}
```

Expected output

```
static: 1001 1002 1003
auto : 1001 1001 1001
1001 1002 1003
```

For the global two-file version, define the variable exactly once in one .c file and declare it with `extern` in the other. Calls to the global function then produce 1001, 1002, 1003, etc. Do not put a second definition in another source file.

5.2 Two independent counters

Static-local version: create two separate functions, such as `next_veg()` and `next_nonveg()`, each with its own static variable; or use a counter structure passed to a function. Global version: define two globals, `veg_token` and `nonveg_token`, with matching extern declarations. Automatic version cannot retain a count between calls, so it must receive a counter pointer or use caller-managed state. A register loop variable is only a temporary loop index; it is not a persistent token counter.

5.3 Comparison

auto: block scope, exists during the block/function call, indeterminate initial value unless initialized, recreated on each call. **static local:** block scope, entire program lifetime, initialized once (zero by default), retains its value. **global:** file scope (extern can expose it elsewhere), entire program lifetime, zero-initialized by default, any code with access can modify it. **register:** block scope and automatic lifetime; requests fast access but compiler may ignore the hint. The address-of operator cannot be applied to a register-declared object in standard C.

Argument evaluation order

In `printf("%d %d %d", next_token(), next_token(), next_token())`, C does not specify the order in which function arguments are evaluated. Therefore the three calls may execute in a different order from their printed argument positions, so the displayed sequence may not be 1001 1002 1003 from left to right. Use separate statements when a predictable order is required.

Alternative: caller-owned counter pointer

```
int next_token_ptr(int *counter)
{
```

```
    return ++(*counter);
}
/* int veg = 1000, nonveg = 1000;
   next_token_ptr(&veg);
   next_token_ptr(&nonveg); */
```

Each call is O(1). The counter lives wherever the caller declares it and remains valid for that object's lifetime. Passing a pointer uses O(1) extra space and naturally supports independent counters without global state. Static/global counters also have O(1) call time but have shared program lifetime and can create hidden dependencies.

TASK 6 – DESIGN CHALLENGE: CANTEEN INVENTORY AND BILLING

Stage 1 – Entities and testable requirements

Entities: Date; PackagedItem (price and expiry); CookedItem (price and preparation time); ComboItem (up to three component item IDs); Item (tagged union of the three item types); StockRecord (item and quantity); SaleLine (item ID, quantity, line revenue); DailySummary.

Testable rules: (1) A sale is accepted only when requested quantity is positive and available stock is sufficient. (2) Accepted sales reduce stock by the sold quantity. (3) A rejected sale leaves stock unchanged. (4) Packaged items with expiry date from today through two days ahead are warned about. (5) A combo contains no more than three component IDs. (6) Summary lines are sorted by revenue in descending order.

Stage 2 and 3 – Data model and design decisions

Use a Date nested structure, an Item structure with an enum tag and union payload, an array of Item records, and an array of SaleLine records. Update stock only after validating the item and quantity. Compare dates using a helper that converts dates to an ordered day number (or a proper date library in production). No global data is necessary: pass arrays and counts to functions. Static storage is appropriate only if a persistent process-wide counter is specifically required.

Stage 4 and 5 – Working demonstration program

```
#include <stdio.h>
#include <string.h>

#define MAX_ITEMS 10
#define MAX_SALES 20

typedef struct { int day, month, year; } Date;
typedef enum { PACKAGED, COOKED, COMBO } ItemKind;

typedef struct {
    float price;
    Date expiry;
} Packaged;
typedef struct {
    float price;
    int preparedHour, preparedMinute;
} Cooked;
typedef struct {
    float price;
    int componentIds[3];
    int componentCount;
} Combo;

typedef struct {
    int id;
    char name[30];
    int stock;
    ItemKind kind;
    union {
        Packaged packaged;
        Cooked cooked;
        Combo combo;
    } data;
} Item;

typedef struct { int itemId, quantity; float revenue; } SaleLine;

int date_key(Date d) { return d.year * 10000 + d.month * 100 + d.day; }

Item *find_item(Item items[], int n, int id)
{
    for (int i = 0; i < n; ++i)
        if (items[i].id == id) return &items[i];
    return NULL;
}

int sell(Item items[], int n, SaleLine sales[], int *saleCount,
         int id, int quantity)
{
    Item *it = find_item(items, n, id);
    if (it == NULL || quantity <= 0 || it->stock < quantity ||
        *saleCount >= MAX_SALES) return 0;

    float price = it->kind == PACKAGED ? it->data.packaged.price :
                  it->kind == COOKED ? it->data.cooked.price :
                                       it->data.combo.price;
    it->stock -= quantity;
    sales[*saleCount].itemId = id;
    sales[*saleCount].quantity = quantity;
```

```
    sales[*saleCount].revenue = price * quantity;
    ++(*saleCount);
    return 1;
}

void expiry_warning(const Item items[], int n, Date today)
{
    int now = date_key(today);
    for (int i = 0; i < n; ++i) {
        if (items[i].kind == PACKAGED) {
            int days = date_key(items[i].data.packaged.expiry) - now;
            if (days >= 0 && days <= 2)
                printf("Expiry warning: %s (within 2 days)\n",
                       items[i].name);
        }
    }
}

void sort_sales(SaleLine sales[], int n)
{
    for (int i = 0; i < n - 1; ++i) {
        int best = i;
        for (int j = i + 1; j < n; ++j)
            if (sales[j].revenue > sales[best].revenue) best = j;
        if (best != i) {
            SaleLine t = sales[i];
            sales[i] = sales[best];
            sales[best] = t;
        }
    }
}

int main(void)
{
    Item items[3] = {
        {1, "Milk", 10, PACKAGED, {.packaged = {30.0f, {12,11,2026}}}},
        {2, "Sandwich", 5, COOKED, {.cooked = {50.0f, 10,30}}},
        {3, "MealCombo", 4, COMBO, {.combo = {100.0f, {1,2,0}, 2}}}
    };
    SaleLine sales[MAX_SALES];
    int saleCount = 0;
    Date today = {11,11,2026};

    printf("Milk sale: %s\n",
           sell(items, 3, sales, &saleCount, 1, 2) ? "accepted" : "rejected");
    printf("Excess sale: %s\n",
           sell(items, 3, sales, &saleCount, 2, 8) ? "accepted" : "rejected");
    printf("Combo sale: %s\n",
           sell(items, 3, sales, &saleCount, 3, 1) ? "accepted" : "rejected");

    expiry_warning(items, 3, today);
    sort_sales(sales, saleCount);
    printf("Daily summary (highest revenue first):\n");
    for (int i = 0; i < saleCount; ++i)
        printf("Item %d qty %d revenue %.2f\n",
               sales[i].itemId, sales[i].quantity, sales[i].revenue);
    printf("Remaining stock: Milk=%d Sandwich=%d MealCombo=%d\n",
           items[0].stock, items[1].stock, items[2].stock);
    return 0;
}
```

Sample output

```
Milk sale: accepted
Excess sale: rejected
Combo sale: accepted
Expiry warning: Milk (within 2 days)
Daily summary (highest revenue first):
Item 3 qty 1 revenue 100.00
Item 1 qty 2 revenue 60.00
Remaining stock: Milk=8 Sandwich=5 MealCombo=3
```

Implementation note: The sample date comparison uses a compact YYYYMMDD key and is suitable for the demonstrated dates, but a production system should use validated calendar dates and proper date arithmetic (including month/year boundaries). Combo pricing is stored as a fixed price in this demonstration; a real canteen may calculate it from component prices and validate component stock.

Linked list versus array

With a linked list, inserting a new item at the head is O(1); removing a known node is O(1) once its predecessor/node is available, but searching by ID is O(n). Removing by ID also requires O(n) search. An array supports O(1) indexed access and good cache locality; searching unsorted IDs is O(n), while insertion/deletion may require shifting O(n) records. A linked list uses extra memory for one or more next pointers per node plus allocation overhead and has poorer locality. An array may reserve unused capacity but avoids per-node pointer overhead.

SUBMISSION CHECKLIST

• Create and test a separate C source file for each required task.

• Include every completed “Complete the Code” program and its compiler output.

• Add input files and output screenshots to the repository.

• Write a README.md with compiler instructions, file descriptions and run commands.

• Include the diagnosis, structure definitions, memory-layout discussion, pseudocode, complexity comparisons and test evidence in the report.

• Add the GitHub repository link to the final submission; the assignment marks it mandatory.

Important: Verify sizeof results and compiler output on the same compiler/platform used for your lab submission. Memory layout, padding and printed addresses can vary between systems.